



বাংলাদেশ আর্মি ইন্টারন্যাশনাল ইউনিভার্সিটি অব সায়েন্স এন্ড টেকনোলজি, কুমিল্লা BANGLADESH ARMY INTERNATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY (BAIUST), CUMILLA

Department of Computer Science and Engineering

Lab Report

Lab Report No : 03			
Lab Report Name : A* Search Algorithm (Romania Road Map Problem)			
Course Title : Artificial Intelligence Sessional			
Course Code : CSE 422			
Name : MD Rakibul Hassan			
ID : 0822220105101024			
Level : 4	Term : 2	Section : A	Group : G1
Date of Submission : 17/05/2026	Semester : Spring	Year : 2026	

Marking Rubric

Problem Understanding & Report Clarity (3)	Implementation (4)	Results, Analysis & Presentation (3)	Total (10)

Key Learnings:

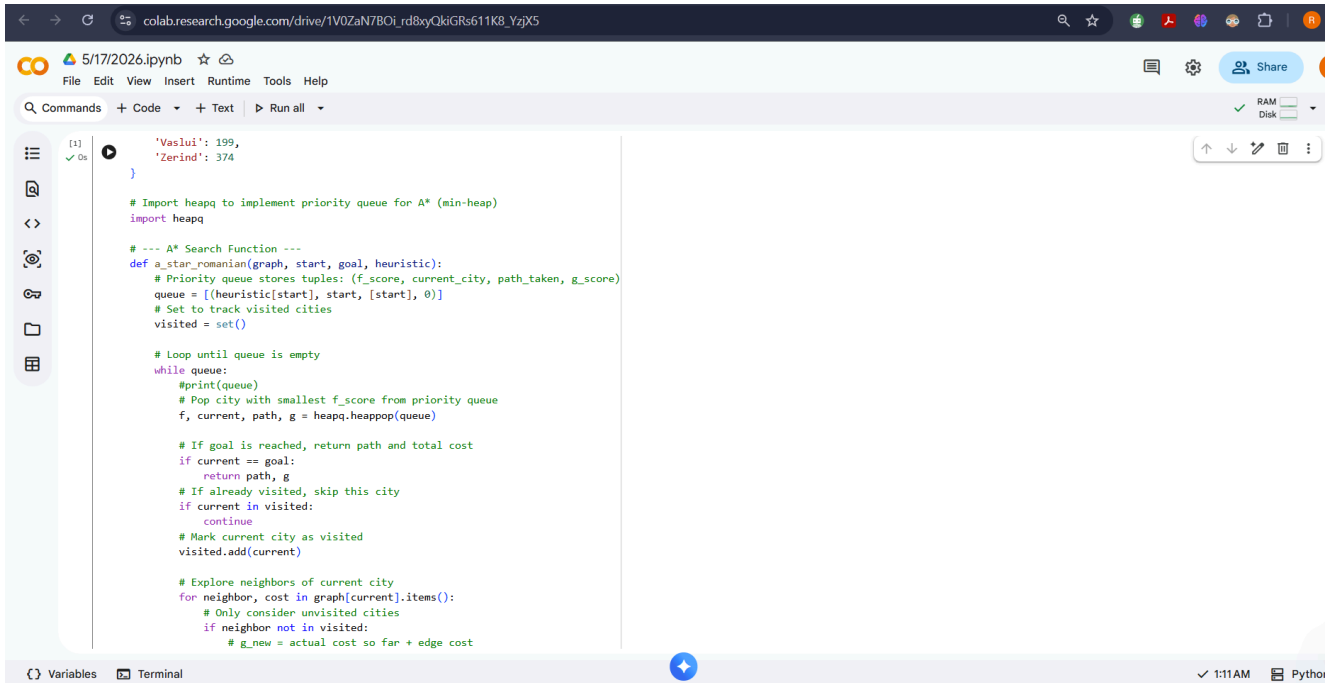
- Learned about the A* search algorithm with hands-on experience.
- Understand the working process of the A* search algorithm, how it combines actual and heuristic values to reach the goal quickly.

Code Implementation & Sample Input/Output:

```
#Informed Search Algorithms: A* Search (Romania Road Map Problem Search)
# --- Graph definition: each city has neighbors with distances (road distances in km) ---
graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97, 'Craiova': 146},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101, 'Craiova': 138},
    'Craiova': {'Rimnicu Vilcea': 146, 'Pitesti': 138, 'Drobeta': 120},
    'Drobeta': {'Craiova': 120, 'Mehadia': 75},
    'Mehadia': {'Drobeta': 75, 'Lugoj': 70},
    'Lugoj': {'Mehadia': 70, 'Timisoara': 111},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}

# --- Straight-Line Distance heuristic: estimated distance from each city to Bucharest ---
heuristic = {
    'Arad': 366,
    'Bucharest': 0,
    'Craiova': 160,
    'Drobeta': 242,
    'Eforie': 161,
```

Figure No: 01



The screenshot shows a Jupyter Notebook interface with a code cell containing the initial part of an A* search function. The code defines a function `a_star_romanian` that takes a graph, start, goal, heuristic, path_taken, and g_score as arguments. It initializes a priority queue, a visited set, and enters a loop to explore neighbors. The code is as follows:

```
[1] ✓ 0s
# Import heapq to implement priority queue for A* (min-heap)
import heapq

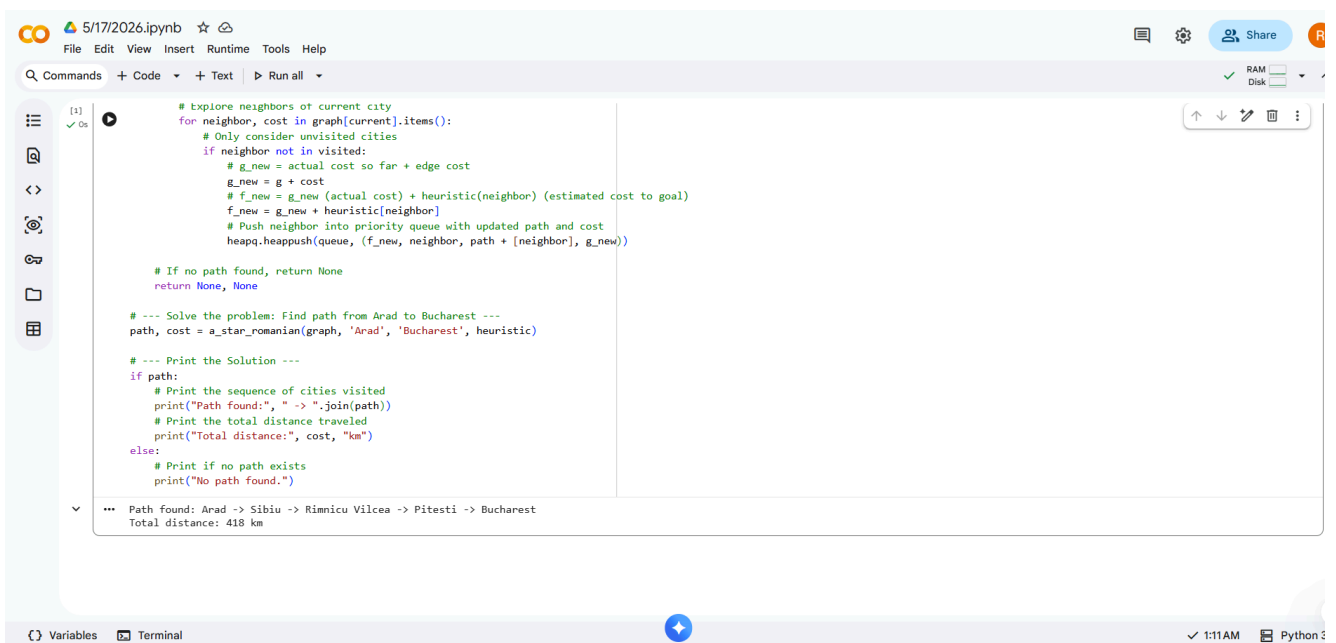
# --- A* Search Function ---
def a_star_romanian(graph, start, goal, heuristic):
    # Priority queue stores tuples: (f_score, current_city, path_taken, g_score)
    queue = [(heuristic[start], start, [start], 0)]
    # Set to track visited cities
    visited = set()

    # Loop until queue is empty
    while queue:
        # Print(queue)
        # Pop city with smallest f_score from priority queue
        f, current, path, g = heapq.heappop(queue)

        # If goal is reached, return path and total cost
        if current == goal:
            return path, g
        # If already visited, skip this city
        if current in visited:
            continue
        # Mark current city as visited
        visited.add(current)

        # Explore neighbors of current city
        for neighbor, cost in graph[current].items():
            # Only consider unvisited cities
            if neighbor not in visited:
                # g_new = actual cost so far + edge cost
```

Figure No: 02



The screenshot shows the same Jupyter Notebook with the completed A* search function and its execution output. The code continues from the previous figure, adding logic to push neighbors into the priority queue and return the final path and cost. The output of the function is displayed below the code cell.

```
    # If no path found, return None
    return None, None

# --- Solve the problem: Find path from Arad to Bucharest ---
path, cost = a_star_romanian(graph, 'Arad', 'Bucharest', heuristic)

# --- Print the Solution ---
if path:
    # Print the sequence of cities visited
    print("Path found:", " -> ".join(path))
    # Print the total distance traveled
    print("Total distance:", cost, "km")
else:
    # Print if no path exists
    print("No path found.")

*** Path found: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total distance: 418 km
```

Figure No: 03

Result Analysis

- The A* search algorithm is one of the popular techniques that are used for path-finding and graph traversal.
- A* search is an informed search algorithm that ignores the visited track to avoid the same search repeatedly.
- A* search successfully gets the shortest path from Arad to Bucharest, which is optimal.